Dataset Preparation

1. **Dataset**: Choose a simple dataset from Kaggle, such as "House Prices - Advanced Regression Techniques" or "Mall Customers." This provides a manageable dataset with both numerical and categorical data.
2. **Tools**: You will need libraries like pandas, numpy, scipy, seaborn, and matplotlib. For advanced techniques like DBSCAN, Isolation Forest, and One-Class SVM, You'll also need sklearn.

Exercise 1: Detecting Outliers with Z-Score

1. **Goal**: Use Z-score to detect outliers in a numerical feature.
2. **Instructions**:
    - Use scipy.stats.zscore to calculate the Z-score for each value in a chosen column (e.g., 'Age' or 'Income').
    - Identify outliers where the absolute Z-score is greater than 3.
    - Try different thresholds (e.g., 2 or 2.5) to see how the number of detected outliers changes.
    - **Optional part: visualization**:
        - Use matplotlib.pyplot.hist to plot a histogram of the feature before and after removing outliers.
        - Overlay a vertical line at Z-scores ±3 using plt.axvline to clearly show the threshold for outliers.
        - Create a seaborn.boxplot to display the feature with and without outliers for a side-by-side comparison.

Exercise 2: Detecting Outliers with Interquartile Range (IQR)

1. **Goal**: Use the IQR method to detect and remove outliers.
2. **Instructions**:
    - Calculate Q1 and Q3 for a feature using numpy.percentile.
    - Define the IQR as IQR = Q3 - Q1.
    - Identify outliers as any point outside [Q1 - 1.5 * IQR, Q3 + 1.5 * IQR].
    - Experiment results with different multipliers other than 1.5 to see its effect on outlier detection.
    - **Optional part: visualization**:
        - Use seaborn.boxplot for the feature before and after removing outliers.
        - Mark the IQR range using plt.axhline at Q1 and Q3 on the box plot, then shade the area between these lines to represent non-outliers.
        - Plot a scatter plot using plt.scatter, marking outliers in a different color to show their position relative to other points.

Exercise 3: Using KNN for Outlier Detection

1. **Goal**: Detect outliers based on distance to nearest neighbors.
2. **Instructions**:
    - Use sklearn.neighbors.NearestNeighbors with a small number of neighbors, such as 5.
    - Calculate the average distance from each point to its nearest neighbors.
    - Set a threshold distance to classify points with high distances as outliers.
    - Try different values for the number of neighbors to see its effect on results.
    - **Optional part: visualization**:
        - Create a scatter plot with two features on the X and Y axes using plt.scatter.
        - Use different colors or markers to distinguish outliers from non-outliers.
        - Overlay contours using seaborn.kdeplot for non-outliers to visualize density, helping students see where outliers fall outside the dense areas.

Optional Exercise 1: Outlier Detection with DBSCAN

1. **Goal**: Use DBSCAN, a density-based clustering method, to identify outliers.
2. **Instructions**:
    - Apply sklearn.cluster.DBSCAN, specifying eps for maximum distance between points and min_samples for the minimum number of points in a cluster.
    - DBSCAN assigns points to clusters; label points with -1 as outliers.
    - Suggest experimenting with different eps values to explore how the density threshold affects outlier classification.
    - **Optional part: visualization:**
        - Plot data points in a scatter plot using plt.scatter, with cluster members in one color and outliers in another.
        - Add a legend to distinguish outliers from clusters.
        - Add text annotations to show the DBSCAN parameters (like eps and min_samples) used, so the effect of parameter tuning is clear.

Optional exercise 2: Isolation Forest for Outlier Detection

1. **Goal**: Use Isolation Forest, an ensemble method, to detect anomalies.
2. **Instructions**:
    - Train an sklearn.ensemble.IsolationForest model on the data, specifying a contamination rate to determine the proportion of outliers.
    - Predict anomaly scores for each data point, classifying those with high scores as outliers.
    - Clarify that Isolation Forest creates splits to isolate points; points that get isolated quickly are more likely to be outliers.
    - Suggest testing different contamination values to see how the predicted outliers change.
    - **Optional part: visualization**:
        - Use a scatter plot with plt.scatter, coloring points by their anomaly scores.
        - Add a color gradient (e.g., using cmap=plt.cm.viridis) to show how the model assigns anomaly scores.
        - Encourage adding a color bar with plt.colorbar to help interpret the gradient values visually.

Optional exercise 3: Outlier Detection with One-Class SVM

1. **Goal**: Detect outliers using One-Class SVM, a type of support vector machine for anomaly detection.
2. **Instructions**:
    - Fit an sklearn.svm.OneClassSVM model on the data, adjusting nu for sensitivity to outliers.
    - Use the predict method to label data points, with -1 indicating outliers.
    - Adjust the nu parameter to explore its effect on the sensitivity of outlier detection.
    - **Optional Part: visualization**:
        - Plot a scatter plot with plt.scatter using two features as axes.
        - Use color coding to distinguish inliers from outliers based on model predictions.
        - Encourage drawing the decision boundary using plt.contour to help students understand how the SVM model classifies data as inliers or outliers visually.